

Final Exam Study Guide

Contents

- Chapter 8 — Decrease-by-Half (Divide and Conquer)
- Chapter 9 — Randomization
- Chapter 10 — Reduction
- Chapter 11 — Dynamic Programming
- Chapter 12 — Lower Bounds
- Chapter 13 — Easy and Impossible Problems
- Chapter 14 — NP-Hard Problems
- Parallel Algorithms
- Quick-Reference Table of Efficiency Classes
- Types of Exam Questions to Expect
- Study Strategy

Final Exam Study Guide — Chapters 8-14 + Parallel Algorithms

110-minute limit · one page of notes allowed · non-cumulative (covers Ch 8-14 only) · bring your CWID

Additions from the lecture slides are marked **[Slides]**. Struck-through topics are **not on the exam**: American Change Making; Ch. 12.6 Reduction Arguments; the parallel merge step itself. You will **not** be asked to prove an NP-complete reduction.

Chapter 8 — Decrease-by-Half (Divide and Conquer)

8.1 The Big Idea

Solve a problem by reducing it to a smaller instance of the same problem, then combining the sub-solution(s) into a full solution.

Decrease-by-half specifically shrinks the input by a **constant fraction** each step (not just by 1, as in decrease-by-one patterns like insertion sort).

[Slides] General template (the pattern every decrease-by-half algorithm follows):

```
decrease_by_half(INPUT):
    if INPUT is base case:
        return base case solution
    left_input, right_input = divide INPUT in half
    left_solution = decrease_by_half(left_input)
    right_solution = decrease_by_half(right_input)
    entire_solution = combine left_solution with right_solution
    return entire_solution
```

Three phases every time: **Divide → Recursion → Combine**.

[Slides] Formal definition of algorithm (Def. 6): a process that, given a problem instance, produces a solution, and is (1) clear enough to implement, (2) correct (always produces a correct solution), and (3) terminates (finite time). Decrease-by-half algorithms must satisfy termination explicitly — see pitfall below.

[Slides] Pitfall — infinite recursion: base cases and the combine phase must work together so recursion always bottoms out for every n . Best practice: design so the general-case threshold is $t = 2$, i.e. base cases explicitly handle $n = 0$ and $n = 1$, general case handles $n \geq 2$. Missing the $n = 1$ base case (only handling $n = 0$) causes infinite recursion — e.g. `dbh_sum([7])` splits into `dbh_sum([])` and `dbh_sum([7])` forever, since `half = 1/2 = 0` gives an empty left and an unchanged right.

8.2 Example: Summation

Classic warm-up: split the array in half, recursively sum each half, add the results. Base case: array of size 0 or 1 returns immediately.

[Slides] Final pseudocode:

```
dbh_sum(L):
    n = len(L)
    if n == 0:
        return 0
    if n == 1:
        return L[0]
    half = n / 2
    left = L[:half]
    right = L[half:]
    return dbh_sum(left) + dbh_sum(right)
```

[Slides] Dividing a list in half (Python slice notation): `half = n / 2` (integer truncation), `left = L[:half]`, `right = L[half:]` — each costs $n/2$ steps to copy.

[Slides] Step-count / variable-initialization counting pattern: primitive types = 1 step; a list with k elements = k steps to build.

[Slides] Reality check: `dbh_sum` costs **$O(n \log n)$** by the master method (below), while the naïve iterative sum (`total = 0; for x in L: total += x`) costs only **$O(n)$** and is simpler/easier to analyze. Lesson: decrease-by-half isn't automatically the best choice — compare against the naïve algorithm before assuming divide-and-conquer wins.

8.3 Analyzing Decrease-by-Fraction Algorithms — Master Method

General recurrence form: $T(n) = a \cdot T(n/b) + f(n)$

- a = number of subproblems
- n/b = size of each subproblem (input shrinks by factor b)
- $f(n)$ = work done outside the recursive calls (combining step)

Compare $f(n)$ to $n^{(\log_b a)}$:

1. If $f(n) = O(n^{(\log_b a - \epsilon)}) \rightarrow T(n) = \Theta(n^{(\log_b a)})$ (recursive work dominates)
2. If $f(n) = \Theta(n^{(\log_b a)} \cdot \log^k n) \rightarrow T(n) = \Theta(n^{(\log_b a)} \cdot \log^{(k+1)} n)$ (balanced — usually $k=0$)
3. If $f(n) = \Omega(n^{(\log_b a + \epsilon)})$ (and regularity holds) $\rightarrow T(n) = \Theta(f(n))$ (combining work dominates)

Be prepared to prove the efficiency class using the master method — show the case, compute $\log_b a$, compare growth rates.

[Slides] Formal “master-form recurrence” definition — the precise shape the master theorem requires:

$T(n) = r \cdot T(n/d) + c(n) \quad \forall n \geq t$
 $T(n) \in O(1) \quad \forall n < t$
 where $c(n) \in O(n^k)$; $r, d, t, k \in O(1)$; $r, t \geq 1$; $d > 1$; $k \geq 0$.

Symbol	Meaning	Constraint
r	number of recursive calls	$r \geq 1$
d	divisor of input size	$d > 1$
t	threshold for the general case	$t \geq 1$
k	exponent in the non-recursive work $c(n)$	$k \geq 0$

[Slides] The Master Theorem (equivalent framing, stated directly in r, d, k):

- If $r < d^k \rightarrow T(n) \in O(n^k)$ (case 1)
- If $r = d^k \rightarrow T(n) \in O(n^k \log n)$ (case 2)
- If $r > d^k \rightarrow T(n) \in O(n^{(\log_d r)})$ (case 3)

[Slides] The Master Method — **6-step recipe** to analyze any decrease-by-half algorithm (memorize this order for the exam):

1. Identify the general-case threshold t from the base-case pseudocode (best practice: $t = 2$).
2. Analyze the base case (assume $n < t$): step-count it, state $T(n) \in O(1) \forall n < t$.
3. Step-count the general case (assume $n \geq t$): count steps outside recursive calls, derive the master-form recurrence $T(n) = r \cdot T(n/d) + c(n) \forall n \geq t$.
4. Identify and check the master-form parts: pull out $c(n)$, r , d , t , k ; verify all constraints hold.
5. Compare r vs d^k to identify which of the 3 cases applies.
6. Simplify and state the conclusion in plain $O(\dots)$ form (e.g. $O(n^k \log n)$ with $k=1$ simplifies to $O(n \log n)$).

[Slides] Worked example — `dbh_sum`:

- $t = 2$; base case $T(n) = 4 \in O(1) \forall n < 2$
- General case: $T(n) = 2 \cdot T(n/2) + n + 4 \forall n \geq 2$
- Parts: $c(n) = n + 4 \in O(n) \rightarrow k = 1$; $r = 2$; $d = 2$; $t = 2$ — all constraints check out.
- Compare: $r ? d^k \rightarrow 2 ? 2^1 \rightarrow 2 = 2 \rightarrow$ case 2.
- Conclusion: $T(n) \in O(n^1 \log n) = O(n \log n)$. `dbh_sum` takes **$O(n \log n)$** time.

8.4 Merge Sort

Recursively split array in half, sort each half, merge the two sorted halves.

Recurrence: $T(n) = 2T(n/2) + \Theta(n) \rightarrow$ Case 2 of master method $\rightarrow \mathbf{O(n \log n)}$.

Be able to **trace it by hand** on a small array (show the split tree and the merge steps). Know how to design similar divide-and-conquer algorithms and analyze them the same way.

[Slides] History (context, likely not tested but good grounding): discovered by John von Neumann in 1945; practical in the 1950s–70s for sorting data on tape drives; historically important because it was faster than $O(n^2)$ but slower than $O(n)$, disproving the early assumption that all time complexities are polynomial.

[Slides] The Sorting Problem (formal definition): input: a list U of comparable elements; output: a list S containing the elements of U in non-decreasing order.

[Slides] Key insight — lists of size ≤ 1 are trivially sorted: the empty list `[]` and any single-element list `[a]` vacuously satisfy “non-decreasing order,” so the base case is simply `if n ≤ 1: return U`.

[Slides] Full pseudocode (with merge factored out as its own sub-algorithm):

```
merge_sort(U):
    if n ≤ 1:
        return U
    half = n / 2
    left = U[:half]
    right = U[half:]
    return merge(merge_sort(left), merge_sort(right))
```

[Slides] The Merge sub-problem: input two lists LS and RS , each already in non-decreasing order; output a list S containing all elements of LS and RS in non-decreasing order.

```

merge(LS, RS):
    S = []
    li = 0 # index of next candidate in LS
    ri = 0 # index of next candidate in RS
    while li < len(LS) and ri < len(RS):
        if LS[li] <= RS[ri]:
            S.add_back(LS[li]); li += 1
        else:
            S.add_back(RS[ri]); ri += 1
    return S + LS[li:] + RS[ri:] # append any leftover elements

```

Key insight enabling $O(n)$: because LS and RS are each already sorted, the least remaining element in each list is always at its current front index — no need to search. Step count: $3 + 5n + n = 6n + 3 \in O(n)$. merge takes **$O(n)$** time.

[Slides] Master method analysis of merge_sort: $t = 2$; base $T(n) = 2 \in O(1) \forall n < 2$; general $T(n) = 2 \cdot T(n/2) + 2n + 2 \forall n \geq 2$; parts $r=2, d=2, k=1 \rightarrow$ case 2 $\rightarrow$ **$O(n \log n)$** .

[Slides] Selection Sort vs Merge Sort: Selection Sort is $O(n^2)$; Merge Sort is $O(n \log n)$ and strictly better — Selection Sort is considered obsolete. Always prefer Merge Sort (or another $O(n \log n)$ sort).

[Slides] Binary Search (third worked decrease-by-half example — know it). Preview of Reduction / pre-sorting: sort first, then search faster. Key insight: since L is sorted, compare q to the middle and recurse into only one half ($r = 1$).

```

binary_search(L, s, e, q):
    n = e - s
    if n == 0: return None
    if n == 1: return s if L[s] == q else None
    half = s + (n / 2)
    if q == L[half]: return half
    if q < L[half]: return binary_search(L, s, half, q)
    return binary_search(L, half + 1, e, q)

```

Master method: $r=1, d=2, k=0 \rightarrow$ case 2 $\rightarrow$ **$O(\log n)$** . Fastest efficiency class covered outside $O(1)$ — good benchmark, and a good example of $r = 1$.

Chapter 9 — Randomization

9.1 The Big Idea

Randomization = intentionally designing an algorithm to make random choices. Deterministic algorithm = no random behavior (everything before this chapter). Randomized algorithm = does behave randomly.

Randomization helps when: (1) there are many alternatives, (2) most are good, (3) selecting the best one deterministically would be slow. (Analogy: picking a lunch place downtown — don't waste time finding the optimal one, just pick a reasonable one at random.)

9.2 Generating Random Numbers

[Slides] TRNG (True Random Number Generator): uses a physical entropy source — genuinely unpredictable. PRNG (Pseudo-Random Number Generator): uses a deterministic equation to produce random-seeming values — not philosophically random, but adequate outside cybersecurity. By default, “random” = PRNG.

Operation	Pseudocode	Time
Random integer in $[min, max]$	<code>random_int(min, max)</code>	$O(1)$
Random element of list L	<code>random_choice(L)</code>	$O(1)$
Randomly shuffle list L	<code>random_shuffle(L)</code>	$O(n)$

Expected Time — Three “flavors” of efficiency class: deterministic (always applies, the default — “ $O(n)$ time” means deterministic), amortized (uneven due to lazy operations, but fast in total over a series), expected (unpredictable due to randomization, fast on average).

- Expected value: for random variable X with outcomes $x_1 \dots x_k$ having probabilities $p_1 \dots p_k$: $E[X] = x_1 p_1 + \dots + x_k p_k$.
- $X \in O(f(n))$ expected means $E[X] \in O(f(n))$ — averaged over all random outcomes, the time is $O(f(n))$.
- Preference ranking: $O(1)$ deterministic (best, suitable for real-time) $> O(1)$ amortized $> O(1)$ expected (good, but slowdown is entirely unpredictable).
- Combining rule (be able to apply this): the “expected” flavor is sticky.
 - $O(x)$ expected + $O(y)$ deterministic = $O(x + y)$ expected
 - $O(x)$ expected $\times n = O(xn)$ expected

9.4 The Las Vegas Pattern

A Las Vegas algorithm always returns a **correct** answer; randomness only affects **runtime** (may be slow with small probability). Randomized quick sort is the course's main Las Vegas example: always sorts correctly; expected $O(n \log n)$, worst-case $O(n^2)$.

9.5 Out-of-Place Quick Sort

Divide-by-value, not by-index: unlike merge sort (which splits by position), quick sort's divide phase moves each element into a sub-list based on its value relative to a chosen pivot.

Pivot: an input element chosen as the boundary value. **Partition:** split the unsorted list into less ($<$ pivot), equal ($=$ pivot, includes duplicates), greater ($>$ pivot) — partitioning does not sort, just divides.

Key insight: the equal sub-list never needs sorting (all elements tied = trivially non-decreasing).

Combine phase is just concatenation (not a merge!): since every element in less $<$ every element in equal $<$ every element in greater by construction, the answer is simply `quick_sort(less) + equal + quick_sort(greater)`.

```

quick_sort(U):
    if n ≤ 1:
        return U
    pivot = random_choice(U)
    less = [x for x in U if x < pivot]
    equal = [x for x in U if x == pivot]
    greater = [x for x in U if x > pivot]
    return quick_sort(less) + equal + quick_sort(greater)

```

Be able to **run it by hand** — trace every sub-list created and the result of each combine phase.

Deterministic pivot choice (e.g., always $U[0]$) gives **$O(n^2)$** time — shown by recurrence $T(n) = T(n-1) + 4n + 2$ when the pivot is always the min/max (not master-theorem form; solved by expansion/summation).

Randomized pivot choice fixes this: only 2 of n elements are “extreme” choices, so the probability every pivot across the whole recursion is extreme is astronomically small. So for practical purposes quick sort behaves like $O(n \log n)$, but the precise worst-case bound is still $O(n^2)$.

Claim to know cold: randomized out-of-place quick sort takes **$O(n \log n)$ expected time, and $O(n^2)$ worst-case time.**

9.6 In-Place Quick Sort

Motivation: avoid allocating new less/equal/greater lists — sort within a range of indices $[s, e]$ of the original array using swaps only (no copying).

In-Place Partition problem: given list U and range $[s, e]$, rearrange elements and return pivot index p such that every element in $U[s:p] < U[p]$, and every element in $U[p+1:e] \geq U[p]$.

Mechanism: swap a random pivot to the end (index $e-1$); maintain a “certified less” zone growing from the left (pointer i) and a “certified $\geq$ ” zone growing from the right (pointer j), with a shrinking “todo” zone in between:

- If $U[i] < \text{pivot}$: leave it, advance i .
- Else if $U[j] \geq \text{pivot}$: leave it, retreat j .
- Else: swap $U[i]$ and $U[j]$, advance i and retreat j .

When $i > j$, swap the pivot (at $e-1$) into position i — that’s the final pivot index p . `in_place_partition` runs in **$O(n)$** time.

Be able to **run it by hand** — trace the partition (zones/swaps) and the full sort (finished zones of every partition phase).

Same efficiency class as out-of-place: **$O(n \log n)$ expected, $O(n^2)$ worst-case** — the only difference is space (in-place uses $O(1)$ extra space vs out-of-place’s $O(n)$).

Comparing Sorting Algorithms

Sorting Algorithm	Expected Time	Worst-Case Time	In-Place?
Selection Sort	$O(n^2)$	$O(n^2)$	yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	no
Quick Sort	$O(n \log n)$	$O(n^2)$	yes

Selection Sort (and other $O(n^2)$ sorts: insertion, bubble) is strictly dominated — never use them. Merge Sort and Quick Sort are tied for fastest on average. Merge Sort’s advantage: guaranteed $O(n \log n)$ worst-case. Quick Sort’s advantage: in-place. Best practice: default to Quick Sort; use Merge Sort instead when the $O(n^2)$ worst-case of Quick Sort is unacceptable (e.g., real-time / safety-critical systems).

Chapter 10 — Reduction

10.1 The Big Idea

Solve problem A by transforming (reducing) it to a problem B you already know how to solve, then use B ’s solution/algorithm to solve A . Total cost = cost of the reduction/transformation + cost of solving B .

[Slides] Two variants: reduction to an algorithm, and reduction to a data structure. Template for reduction to an algorithm:

```

reduction_pattern(input_to_problem_A):
    input_to_problem_B = pre-process input_to_problem_A
    solution_to_problem_B = solve_B(input_to_problem_B)
    solution_to_problem_A = post-process solution_to_problem_B
    return solution_to_problem_A

```

If A reduces to B , then A is easier than B (or tied). Binary search’s setup (pre-sorting) is a preview of this pattern. **[Slides]** Informal analysis (after midterm): evidence limited to big- O of bottleneck steps — omit full step counts when obvious.

10.2 Reduction to Sorting

Median finding: sort the array, return the middle element(s). Cost dominated by sort $\rightarrow O(n \log n)$.

[Slides] Set intersection: concatenate $L+R$, sort, then take elements that appear two-in-a-row (common elements appear twice) $\rightarrow O(n \log n)$ expected.

```

set_intersect_sort(L, R):
    sorted = quick_sort(L + R)
    output = []
    for i from 0 to len(sorted) - 2:
        if sorted[i] == sorted[i + 1]:
            output.add(sorted[i])
    return output

```

[Slides] Min-and-max by sorting: sort, return (first, last) $\rightarrow O(n \log n)$ expected. (A linear scan is faster in practice; this is a reduction example.)

10.3 Reduction to Hash Table Operations

[Slides] Know the big picture: a map stores associations (key $\rightarrow$ value). Hashing converts a key into an array index; collisions are inevitable (pigeonhole principle) and handled by chaining or open addressing.

Core operations (insert, delete, lookup) take **$O(1)$ expected time**.

Reduction pattern: naïve algorithms that are $O(n^2)$ (e.g., nested search/count) can be sped up to **$O(n)$ expected** by replacing the bottleneck $O(n)$ search with $O(1)$ expected hash-table ops.

[Slides] Recipe: (1) write naïve, (2) identify bottleneck loop, (3) pre-populate a hash table and replace the slow lookup.

```
set_intersect_hash(L, R):
    hm = HashMap()
    for r in R: hm[r] = True
    S = []
    for x in L:
        if x in hm: S.add(x)
    return S
```

Mode average the same way: map each element to its count, then scan for the max $\rightarrow O(n)$ expected.

10.4 Priority Search Queues; Prim-Jarník and Dijkstra

[Slides] A Priority Search Queue (PSQ) is a priority queue that also supports decreasing a key's priority. With Fibonacci-heap-style bounds from the slides: Insert $O(1)$, RemoveMin $O(\log n)$ amortized, DecreasePriority $O(1)$ amortized.

Revisit Prim-Jarník (MST) and Dijkstra (non-negative single-source shortest paths) as applications of PSQs over graphs. Both run in **$O(m + n \log n)$** with a suitable PSQ (without it: $O(mn)$).

Correspondence idea: Prim keys = edges (priority = weight if bridge, else ∞); Dijkstra keys = vertices (priority = known distance, else ∞). The sequential "find min bridge / closest unseen" search is replaced by RemoveMin / DecreasePriority.

Chapter 11 — Dynamic Programming

11.1 The Big Idea

Break a problem into overlapping subproblems, solve each subproblem once, store (tabulate) the result, and reuse it — avoids the exponential blow-up of naïve recursion. Requires optimal substructure + overlapping subproblems.

[Slides] **Tabulation:** store solutions to problem instances in a table. Base cases initialized immediately; general cases initialized from other table elements; algorithm initializes all table elements; finally the solution is contained in the table.

[Slides] Top-down thinking = recursion / decrease-by-half (break large into small). Bottom-up thinking = dynamic programming (combine small into large).

[Slides] Design process: dimensions (1D/2D/...) $\rightarrow$ table invariant $\rightarrow$ base-case pseudocode $\rightarrow$ general-case pseudocode $\rightarrow$ complete bottom-up draft.

```
dynamic_programming_1D(input):
    A = array that is large enough to avoid out-of-bounds bugs
    Initialize base-case elements of A
    for i from smallest general case to largest general case:
        A[i] = initialize A[i] using elements A[<i]
    return A[solution index]
```

11.2 1D Dynamic Programming and Fibonacci Numbers

Build a table $F[0..n]$ bottom-up instead of recomputing recursively. Runs in **$O(n)$** time. Be able to run it by hand (fill in the table). Be able to design similar 1D DP algorithms: identify the recurrence, base case(s), fill order, and where the answer lives in the table.

[Slides] Fibonacci Number Problem: input integer $n \geq 0$; output $F(n)$. $F(0)=0$, $F(1)=1$, $F(n)=F(n-1)+F(n-2)$ for $n \geq 2$. First ten: 0,1,1,2,3,5,8,13,21,34.

[Slides] Naïve recursion calls itself twice on $n-1$ and $n-2 \rightarrow$ exponential time — extremely slow.

[Slides] Table invariant: $A[i]$ = the i -th Fibonacci number = $F(i)$.

```
fibonacci_dynamic_programming(n):
    A = [0] * (n + 1)
    A[1] = 1
    for i from 2 to n:
        A[i] = A[i-1] + A[i-2]
    return A[n]
```

Analysis: initializing A takes $O(n)$; the for loop takes $O(n) \rightarrow O(n)$ total. Huge speedup versus the exponential recursive algorithm. Slide demo: trace for $n = 5$ (show table A; output 5).

~~11.3 American Change-Making — not on exam (struck through).~~

Edit Distance

2D DP table where $D[i][j]$ = min number of insertions/deletions/substitutions (overwrites) to convert a prefix of length i of string A into a prefix of length j of string B. Runs in **$O(n^2)$** time (table has $\sim n \times m$ cells, $O(1)$ work per cell). Be able to run it by hand (fill the grid, trace the recurrence) and design similar 2D DP algorithms.

[Slides] Formal problem: input start string s and goal string t ; output the edit distance between s and t . Levenshtein operations: overwrite, insert, delete. Example: "diestra" $\rightarrow$ "dijkstra" needs insert "j" + delete "e" $\rightarrow$ distance 2.

[Slides] Dimensions: two ways to shrink the input (shrink s , shrink t) $\rightarrow$ 2D table. Invariant: $A[i][j]$ = edit distance between $s[:i]$ and $t[:j]$.

Base cases: $A[0][j] = j$ (j inserts from empty string); $A[i][0] = i$ (i deletes to empty string).

```

if s[i-1] == t[j-1]:
    A[i][j] = A[i-1][j-1]          # match – no action
else:
    A[i][j] = min(1 + A[i-1][j],   # delete
                  1 + A[i][j-1],   # insert
                  1 + A[i-1][j-1]) # overwrite
# answer at A[s.size][t.size]

```

[Slides] Conclusion: edit_distance takes $O(|s| \times |t|)$ time. Slide demo: trace for $s = \text{"has"}$, $t = \text{"make"}$ (show full table A; output 3).

Chapter 12 — Lower Bounds

12.1 The Big Idea

A lower bound tells you the best possible efficiency class any algorithm could achieve for a problem — it's a statement about the problem, not a specific algorithm.

[Slides] Act I designed algorithms. Act II studies limits: lower bounds ("speed limit"), NP-complete (apparently only exp. exhaustive search), undecidable (literally impossible).

12.2 Notation and Terminology

- **Lower bound Ω** — algorithm/problem takes at least this long.
- **Upper bound O** — algorithm takes at most this long (for a problem: established by exhibiting an algorithm).
- **Tight bound Θ** — matching upper and lower bounds (know it exactly) → algorithm design for this problem is "done / solved science."

Know precisely what it means for a problem (not algorithm) to have a lower bound.

12.3 Proving Negatives

Proving a positive is straightforward (show a receipt / an example). Proving a negative is difficult — "prove you never bought Funyuns" / "prove no algorithm is faster than...". That is why lower-bound proofs feel different from Act I analyses.

12.4 Trivial Lower Bounds

[Slides] Strategy: every correct algorithm must create the output object(s). Analyze the space complexity $\Omega(f(n))$ of the problem output, then conclude the problem has lower bound $\Omega(f(n))$.

Merge example: output list has n elements → Merge has lower bound $\Omega(n)$. Combined with the $O(n)$ merge algorithm → Merge has tight bound $\Theta(n)$.

Sorting's trivial lower bound is only $\Omega(n)$ — that leaves a gap under the $O(n \log n)$ upper bound from merge sort.

12.5 The Tight Bound for Sorting

Comparison-based sorting has a lower bound of $\Omega(n \log n)$, proven via an information-theoretic / decision-tree style argument: $n!$ possible orderings; a binary comparison halves the candidate permutations; need depth $\geq \log_2(n!) = \Theta(n \log n)$ to distinguish all outcomes.

[Slides] Sketch to write on the exam: (1) n distinct elements → $n!$ permutations; (2) each comparison of a vs b eliminates about half the remaining permutations; (3) need $\log_2(n!)$ comparisons; (4) $\log_2(n!) = \sum_i \log_2 i$ has n terms that are $\Omega(\log n)$ each → $\log_2(n!) \in \Omega(n \log n)$; (5) therefore every comparison-based sorting algorithm takes $\Omega(n \log n)$ time.

Combined with merge sort's $O(n \log n)$ upper bound → comparison sorting is $\Theta(n \log n)$ — tight bound. Solved science; no possibility of a faster efficiency class for comparison-based sorting.

~~12.6 Reduction Arguments — not on exam (struck through).~~

Chapter 13 — Easy and Impossible Problems

13.1 The Big Idea

Problems can be classified by how hard they are to solve algorithmically — from efficiently solvable, to hard-but-solvable, to not solvable by any algorithm at all.

13.2 Efficiently-Solvable Problems and P

P = class of problems solvable in polynomial time (worst-case), i.e., $O(n^k)$ for some constant k . "Efficient" is generally equated with "polynomial time" in this context.

[Slides] Polynomial time includes $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(n^3)$. Not polynomial: $O(2^n)$, $O(n!)$. (Lemmas: $\log n \in O(n)$ and $n \log n \in O(n^2)$, so log factors still count as polynomial.)

[Slides] Prove $X \in P$ (template): design any poly-time algorithm (naïve is fine) → analyze → conclude $X \in P$. Example: Pair Sum double loop is $O(n^2)$ → Pair Sum $\in P$.

13.3 Unsolvable Problems and Decidability

Undecidable problems exist — no algorithm can solve them for all inputs, no matter how much time is allowed.

The Halting Problem: no algorithm can determine, for an arbitrary program and input, whether that program will halt or run forever. Classic proof by contradiction/diagonalization.

[Slides] Formal: decidable = some algorithm solves it (any finite time); undecidable = no such algorithm. $P \subseteq \text{Decidable}$.

[Slides] Halting proof sketch: suppose `halts(proc, I)` exists → build `self_halts(proc) = halts(proc, proc)` → build `contrary(proc)` that loops forever if `self_halts` says True, else returns → evaluate `contrary(contrary)` → both cases contradict → `halts` cannot exist. Implication: compilers cannot detect all infinite loops.

Chapter 14 — NP-Hard Problems

14.1 Verifiable Problems and NP

NP = class of problems where a proposed solution ("certificate" / candidate) can be verified in polynomial time, even if finding the solution might take longer. Note: $P \subseteq NP$ (anything solvable in poly time is also poly-time verifiable).

[Slides] Course definition: NP = problems with an exhaustive algorithm whose verifier part runs in polynomial time. Solve = transform input into output; verify = decide whether one candidate is valid.

[Slides] Prove $X \in NP$: define a candidate $\rightarrow$ design the verifier $\rightarrow$ prove poly time $\rightarrow$ conclude $\in NP$. Example: Clique decision — candidate is a vertex set C ; verifier checks $|C|=k$ and every pair has an edge — polynomial $\Rightarrow$ Clique $\in NP$.

14.2 Hard Problems and NP-Hardness

A problem is **NP-hard** if every problem in NP can be reduced to it in polynomial time — i.e., it's at least as hard as the hardest problems in NP. A problem that is both NP-hard and in NP is **NP-complete**.

[Slides] Goldilocks: $\in NP$ (not impossible) + NP-hard (not easy) = "hard." Formal: $E \leq_p H$ means a reduction solves E by calling a solver for H with poly-time pre/post-processing (H harder than E , or tied). H is NP-hard if $\forall E \in NP, E \leq_p H$.

14.3 A First NP-Complete Problem

Know the canonical first NP-complete problem (Circuit-SAT / SAT) and why establishing one NP-complete problem lets others be shown NP-complete by reduction from it.

[Slides] Circuit Satisfaction (CSAT): input a Boolean circuit C (DAG of AND/OR/NOT/OUT/variables); output a satisfying assignment A , or None. Cook-Levin Theorem: CSAT is NP-complete. (1) $\in NP$: candidate = assignment; verifier evaluates the circuit — $O(n^2)$. (2) NP-hard: for any $E \in NP$, compile verify_E into a Boolean circuit (standard model: each CPU/pseudocode step $\rightarrow$ circuit layers); solving CSAT recovers a solution for E ; pre/post-processing is polynomial.

14.4 Proving NP-Completeness by Reduction

General method: show the problem is in NP (verifiable in poly time) + reduce a known NP-complete problem to it in poly time.

You will NOT be asked to prove an NP-complete reduction on the exam — but you should understand what the technique means conceptually, and be able to categorize problems as P / NP-complete / undecidable.

[Slides] Notable NP-complete problems: CSAT, SAT, Clique, Vertex Cover, TSP, Set Partition, and many generalized games (Chess, Go, Sudoku, Mario, ...). Commonalities: in NP (verify fast); hard (exhaustive search / exponential seems necessary; too slow in practice); but useful if solved.

14.5 P versus NP Revisited

The open question: does $P = NP$? Know what's at stake (if $P = NP$, every efficiently verifiable problem would be efficiently solvable) and that it remains unproven either way.

[Slides] We know $P \subseteq NP$. Open: $P \subset NP$ or $P = NP$? (\$1M Clay prize.) If you find a poly-time algorithm for one NP-complete problem $\Rightarrow P = NP$ (all NP get fast algorithms via reduction; crypto breaks). Conjecture/advice from the slides: live as if $P \subset NP$ — treat NP-complete problems as impractical; treat cryptography as secure.

Parallel Algorithms

Overcoming Lower Bounds

Sequential lower bounds (like $\Omega(n \log n)$ for comparison sorting) apply to a single processor. With multiple processors working simultaneously, wall-clock time can be reduced below the sequential lower bound, even though total work doesn't decrease.

[Slides] Act III: sidestep limits. A tight sequential lower bound cannot be broken — but a parallel algorithm can make user wait time smaller while work still respects the bound.

Analyzing Parallel Algorithms — Processors, Work, Span

- **p** = number of processors.
- **Work** = total number of operations across all processors (same as sequential time complexity if run on 1 processor). Analyze work by ignoring parallelism and counting steps as usual.
- **Span** = length of the longest chain of dependent operations / elapsed time start $\rightarrow$ finish (the critical path) — take the **maximum** of parallel branches; sequential steps count as usual.

```
in parallel:
  statement 1
  statement 2
  ...
  statement k
```

Goal: work matches the lower bound; span beats it. Scalable: span $O(X / p)$ for a problem with lower bound $\Omega(X)$.

Embarrassingly Parallel Problems

Problems where subproblems are fully independent (no communication/coordination needed) — trivially easy to parallelize.

[Slides] Formal: a problem is embarrassingly parallel when there is a scalable parallel algorithm with span that is constant with respect to n (e.g., p -processor summation with $p = n$).

Parallel Sorting — $O(n \log n)$ Work, Sublinear Span

A parallel sorting algorithm can achieve the same total work as sequential sorting, $O(n \log n)$, while reducing the span to something sublinear in n (e.g., polylogarithmic) by exploiting parallel merge/partition steps.

Parallel Merge Sort

Parallelize the recursive split-and-sort structure of merge sort across processors.

The parallel merge step itself will **NOT** be on the exam — but the overall parallel merge sort structure/idea is fair game. **[Slides]** With parallel merge: work $O(n \log n)$, span $O(\log^3 n)$. First draft (parallel recurse, sequential merge) has span $O(n)$ — merge is the bottleneck.

Samplesort

A parallel-friendly generalization of quicksort: pick multiple “splitter” samples to partition the data into p buckets in parallel, sort each bucket independently (in parallel), then concatenate — avoids the single-pivot bottleneck of standard quicksort when parallelizing.

[Slides] Oversampling: randomly select $p \cdot k$ finalists, sort them, take every k -th as a splitter (prevents entirely worst-case splitters). Then in parallel, binary-search each element into a bucket and recurse. Work $O(n \log n)$ **expected**; span $O(\log n)$ **expected**. Trade-off vs parallel merge sort: better span, but expected/randomized (merge sort is deterministic).

Quick-Reference Table of Efficiency Classes

Topic	Time Complexity
Merge Sort	$\Theta(n \log n)$ / $O(n \log n)$
Binary Search [Slides]	$O(\log n)$
Out-of-Place Quick Sort	$O(n \log n)$ expected / $O(n^2)$ worst
In-Place Quick Sort	$O(n \log n)$ expected / $O(n^2)$ worst
Reduction to Sorting (median, set intersection)	$O(n \log n)$
Hash table operations	$O(1)$ expected
Naive $\rightarrow$ hash-table-spiced-up algorithms	$O(n^2) \rightarrow O(n)$ expected
Prim-Jarník / Dijkstra (priority search queue)	$O(m + n \log n)$
1D DP (Fibonacci)	$O(n)$
Edit Distance (2D DP)	$O(n^2)$ / $O(s \times t)$
Comparison-based sorting lower bound	$\Omega(n \log n)$, tight $\Theta(n \log n)$
P	Polynomial time solvable
NP	Polynomial time verifiable
Parallel merge sort	$O(n \log n)$ work, $O(\log^3 n)$ span
Samplesort	$O(n \log n)$ work expected, $O(\log n)$ span expected

Types of Exam Questions to Expect

- Define terminology in your own words
- Write a problem definition
- Perform a step count
- Prove an efficiency class (limits or properties of O)
- Analyze pseudocode (step count + efficiency class proof together)
- Trace algorithm execution by hand, showing work
- Design algorithms using decrease-by-half, reduction, or DP patterns
- Justify / prove a lower bound
- Categorize a problem as P, NP-complete, or undecidable
- Prove a problem is in P or in NP

NOT required: proving an NP-complete reduction; the parallel merge step; American Change-Making; Reduction Arguments (12.6).

Study Strategy

- Redo Problem Set questions from “Decrease-By-Half” onward — exam questions mirror their style.
- For each algorithm, practice: (1) tracing it by hand on a small example, (2) writing its recurrence or step count, (3) stating its efficiency class with justification.
- Memorize the **6-step Master Method** process cold (threshold $\rightarrow$ base case $\rightarrow$ general case $\rightarrow$ identify $r, d, t, k \rightarrow$ compare r vs $d^k \rightarrow$ simplify) — it’s the fastest way to justify divide-and-conquer efficiency classes under time pressure. **[Slides]**
- Practice the $\Omega(n \log n)$ sorting lower bound proof (information-theoretic / decision-tree argument) — a favorite “prove a lower bound” question.
- Be able to explain, in plain language, the difference between P, NP, NP-hard, NP-complete, and undecidable — a likely short-answer topic.
- **[Slides]** Be comfortable hand-tracing the canonical examples end-to-end: `dbh_sum`, `merge_sort` (with merge factored out), `binary_search`, out-of-place and in-place quick sort, Fibonacci table, and an edit-distance grid — a “design/analyze a similar algorithm” question will likely resemble one of these.
- Compress essentials onto your **one allowed page** of notes (both sides OK). Write your name on the notes if you want them returned. Bring your **CWID**.